

LISTA DE EXERCÍCIOS – PONTEIROS

PONTEIROS

1. O que é um ponteiro? E para que serve um ponteiro?
2. Quais das seguintes instruções são corretas para declarar um ponteiro?
a) `int _ptr x;`
b) `int *ptr;`
c) `*int ptr;`
d) `*x;`
3. Qual é a maneira correta de referenciar **ch**, assumindo que o endereço de **ch** foi atribuído ao ponteiro **indica**?
a) `*indica;`
b) `int *indica;`
c) `*indic;`
d) `ch`
e) `*ch;`
4. Na expressão `float *pont;` o que é do tipo float?
a) a variável `pont`.
b) o endereço de `pont`.
c) a variável apontada por `pont`.
d) nenhuma das anteriores.
5. Assumindo que o endereço de **num** foi atribuído a um ponteiro **pnum**, quais das seguintes expressões são verdadeiras?
a) `num == &pnum`
b) `num == *pnum`
c) `pnum == *num`
d) `pnum == &num`
6. Assumindo que queremos ler o valor de **x**, e o endereço de **x** foi atribuído a **px**, a instrução seguinte é correta? Por que?
`scanf ( "%d", *px );`
7. Qual é a instrução que deve ser adicionada ao programa seguinte para que ele trabalhe corretamente?

```
main ( ) {  
    int j, *pj;  
    *pj = 3;  
}
```
8. Assumindo que o endereço da variável **x** foi atribuído a um ponteiro **px**, escreva uma expressão que não usa **x** e divida **x** por 5.
9. Qual o valor das seguintes expressões:

```
int i = 3, j = 5;  
int *p = &i, *q = &j;
```


a) `p == &i`
b) `*p - *q`
c) `**&p`
10. Qual será a saída deste programa?

```
main ( ) {  
    int i = 5, *p = &i;  
    printf ( "%u %d %d %d %d \n", p, *p+2, **&p, 3**p, **&p+4 );  
}
```

11. Se *i* e *j* são variáveis inteiras e *p* e *q* são ponteiros para inteiros, quais das seguintes expressões de atribuição são ilegais?

- | | |
|--------------------------------|---|
| a) <i>p</i> = & <i>i</i> ; | e) <i>i</i> = *&*& <i>j</i> ; |
| b) *& <i>q</i> = & <i>j</i> ; | f) <i>q</i> = & <i>p</i> ; |
| c) <i>p</i> = *&*& <i>i</i> ; | g) <i>i</i> = (* <i>p</i>) ++ + * <i>q</i> ; |
| d) <i>i</i> = (*&)& <i>j</i> ; | |

12. Seja a seguinte seqüência de instruções em um programa C:

```
int *pti;
int i = 10;
pti = &i;
```

Qual afirmativa é **falsa**?

- a) *pti* armazena o endereço de *i*
- b) **pti* é igual a 10
- c) ao se executar **pti* = 20; *i* passará a ter o valor 20
- d) ao se alterar o valor de *i*, **pti* será modificado
- e) *pti* é igual a 10

13. Considerando as variáveis e ponteiros definidos abaixo; quais são as atribuições permitidas?

```
int    x, *ptx, **pp;
float  a, *pta, **pf;
```

- | | | |
|--------------------------------|-------------------------------|-------------------------------|
| a) <i>x</i> = 100; | e) <i>pp</i> = & <i>pta</i> ; | i) <i>pp</i> = & <i>x</i> ; |
| b) * <i>pta</i> = & <i>a</i> ; | f) ** <i>pf</i> = 7.9; | j) <i>pf</i> = & <i>pta</i> ; |
| c) <i>ptx</i> = & <i>a</i> ; | g) * <i>ptx</i> = 20; | |
| d) * <i>pf</i> = & <i>a</i> ; | h) <i>ptx</i> = & <i>x</i> ; | |

14. Considerando as variáveis e ponteiros definidos abaixo; quais são as atribuições permitidas?

```
int    i, *pi, **ppi;
float  f, *pf, **ppf;
```

- | | | |
|---------------------------------|-----------------------------|-------------------------------|
| a) <i>i</i> = <i>f</i> ; | e) * <i>pf</i> = 10; | i) <i>ppf</i> = & <i>pf</i> ; |
| b) <i>pf</i> = & <i>i</i> ; | f) <i>f</i> = <i>i</i> ; | j) ** <i>ppi</i> = 100; |
| c) * <i>pf</i> = 5.9; | g) <i>pi</i> = & <i>f</i> ; | |
| d) * <i>ppi</i> = & <i>pi</i> ; | h) * <i>pi</i> = 7.3; | |

15. Dadas as declarações abaixo; qual é o valor dos itens:

```
int  x = 10, *px = &x, **ppx = &x;
float y = 5.9, *py = &y, **ppy = &y;
```

x FFA0	y FFB4	px FFFO	py FFC6	ppy FFA6	ppx FFD4
--	--	---	---	--	--

- | | | |
|------------------|---------------------|-----------------------|
| a) <i>x</i> = | g) * <i>ppx</i> = | m) & <i>ppy</i> = |
| b) * <i>py</i> = | h) <i>py</i> = | n) *& <i>px</i> = |
| c) <i>px</i> = | i) & <i>x</i> = | o) ** <i>ppx</i> ++ = |
| d) & <i>y</i> = | j) <i>py</i> ++ = | p) <i>px</i> ++ = |
| e) * <i>px</i> = | k) * <i>px</i> -- = | q) & <i>ppx</i> = |
| f) <i>y</i> = | l) ** <i>ppy</i> = | |

16. Quais serão os valores de x, y e p ao final do trecho de código abaixo?

```
int x, y, *p; y = 0;
p = &y;
x = *p;
x = 4;
(*p)++;
--x;
(*p) += x;
```

17. Os programas (trechos de código) abaixo possuem erros. Qual(is)? Como deveriam ser?

a)

```
void main() {
    int x, *p;
    x = 100;
    p = x;
    printf("Valor de p: %d.\n", *p);
}
```

b)

```
void troca (int *i, int *j) {
    int *temp;
    *temp = *i;
    *i = *j;
    *j = *temp;
}
```

c)

```
char *a, *b;
a = "abacate";
b = "uva";
if (a < b)
    printf ("%s vem antes de %s no dicionário", a, b);
else
    printf ("%s vem depois de %s no dicionário", a, b);
```

18. Suponha que os elementos do vetor v são do tipo int e cada int ocupa 8 bytes no seu computador. Se o endereço de v[0] é 55000, qual o valor da expressão v + 3?

19. Escreva uma função mm que receba um vetor inteiro v[0..n-1] e os endereços de duas variáveis inteiras, digamos min e max, e deposite nessas variáveis o valor de um elemento mínimo e o valor de um elemento máximo do vetor. Escreva também uma função main que use a função mm.

20. Suponha que v é um vetor. Descreva a diferença conceitual entre as expressões v[3] e v + 3.

21. (sem usar o computador) Qual o conteúdo do vetor a depois dos seguintes comandos.

```
int a[99];
for (i = 0; i < 99; ++i)
```

```

    a[i] = 98 - i;
for (i = 0; i < 99; ++i)
    a[i] = a[a[i]];

```

22. Escreva uma função chamada troca que troca os valores dos parâmetros recebidos. Sua assinatura deve ser:

```
void troca(float *a, float *b);
```

23. Crie uma função que receba uma string como parâmetro (de tamanho desconhecido) e retorne uma cópia da mesma. A assinatura da função deve ser:

```
char *strcpy(char *str);
```

24. Escreva uma função que recebe como parâmetros um vetor de inteiros v, o número de elementos dele N e ponteiros para variáveis nas quais devem ser armazenados os valores máximo e mínimo do vetor. Sua assinatura deve ser:

```
void maximoMinimo(int *v, int N, int *maximo, int *minimo);
```

25. Qual o resultado do código abaixo? Explique cada linha.

```

int x = 100, *p, **pp;
p = &x;
pp = &p;
printf("Valor de pp: %d\n", **pp);

```

26. O que imprime o programa a seguir? Tente entendê-lo e responder. A seguir, execute-o e comprove o resultado.

```

#include <stdio.h>
int main() {
    int t, i, M[3][4];
    for (t=0; t<3; ++t)
        for (i=0; i<4; ++i)
            M[t][i] = (t*4)+i+1;
    for (t=0; t<3; ++t) {
        for (i=0; i<4; ++i)
            printf ("%3d ", M[t][i]);
        printf ("\n");
    }
    return(0);
}

```

27. Responda as perguntas abaixo. Se achar necessário, faça testes no programa em C.

a) Explique a diferença entre

p++; (*p)++; *(p++);

b) O que quer dizer *(p+10);?

c) Explique o que você entendeu da comparação entre ponteiros.

28. Qual o valor de y no final do programa? Tente primeiro descobrir e depois verifique no computador o resultado. A seguir, escreva um /*comentário */ em cada comando de

atribuição explicando o que ele faz e o valor da variável à esquerda do '=' após sua execução.

```
int main() {
    int y, *p, x;
    y = 0;
    p = &y;
    x = *p;
    x = 4;
    (*p)++;
    x;
    (*p) += x;
    printf ("y = %d\n", y);
    return(0);
}
```

29. Reescreva o programa abaixo usando ponteiros

```
int main () {
    float matr[50][50];
    int i,j;
    for (i=0;i<50;i++)
        for (j=0;j<50;j++)
            matr[i][j]=0.0;
    return(0);
}
```

30. Diga quais expressões abaixo são válidas ou não. Considere as declarações

```
int vetor[10];
int *ponteiro;
a) vetor = vetor + 2;
b) vetor++;
c) vetor = ponteiro;
d) ponteiro = vetor;
e) ponteiro = vetor+2;
```

31. Explique o que faz o programa abaixo

```
main(){
    float vet[5] = {1.1,2.2,3.3,4.4,5.5};
    float *f;
    int i;
    f = vet;
    printf("contador/valor/valor/endereco/endereco");
    for(i = 0 ; i <= 4 ; i++){
        printf("\ni = %d",i);
        printf(" vet[%d] = %.1f",i, vet[i]);
        printf(" *(f + %d) = %.1f",i, *(f+i));
        printf(" &vet[%d] = %X",i, &vet[i]);
        printf(" (f + %d) = %X",i, f+i);
    }
}
```

}

32. Assumindo que `M1[]` é um vetor do tipo `int`, quais das seguintes expressões referenciam o valor do terceiro elemento de `M1`?

- a) `*(M1 + 2)` b) `*(M1 + 4)` c) `M1 + 4` d) `M1 + 2`

33. Considere a declaração:

```
int mat[4], *p, x;
```

Quais expressões são válidas? Justifique:

- a) `p = mat + 1;` b) `p = mat++;` c) `p = ++mat;` d) `x = (*mat)++;`

34. Faça um programa usando ponteiros, para ordenar 5 números e mostra-los ordenados na tela.

35. Explique o que o programa abaixo faz

```
main() {  
    int vet[] = {4,9,12};  
    int i,*ptr;  
    ptr = vet;  
    for(i = 0 ; i < 3 ; i++) {  
        printf("%d ",*ptr++);  
    }  
}
```

36. Seja `vet` um vetor de 4 elementos: TIPO `vet[4]`. Supor que depois da declaração, `vet` esteja armazenado no endereço de memória 4092 (ou seja, o endereço de `vet[0]`). Supor também que na máquina usada uma variável do tipo `char` ocupa 1 byte, do tipo `int` ocupa 2 bytes, do tipo `float` ocupa 4 bytes e do tipo `double` ocupa 8 bytes. Qual o valor de `vet+1`, `vet+2` e `vet+3` se:

- a) `vet` for declarado como `char`?
b) `vet` for declarado como `int`?
c) `vet` for declarado como `float`?
d) `vet` for declarado como `double`?

37. Considere um micro cujo barramento de endereços possui 16 bits. Considere um ponteiro `p` apontando para a primeira posição de memória da figura abaixo.

120A	AB
120B	01
	FF
	ED
	34
	4C

Responda:

qual o valor de `p`?

qual o valor de `*p`?

qual o valor de `&p`?

qual o valor de `*(p+1)`?

qual o valor de (p+5)?
qual o valor de (p+A)?
qual o valor de *(p+2)?

38. Verifique o programa abaixo. Encontre o seu erro e corrija-o para que escreva o numero 10 na tela.

```
#include <stdio.h>
int main() {
    int x, *p, **q;
    p = &x;
    q = &p;
    x = 10;
    printf("\n%d\n", &q);
    return(0);
}
```

39. Qual a saída do seguinte trecho de código?

```
1. #include <stdio.h>
2. int main (int argc, char* argv[]) {
3.     char *p = "Prata";
4.     printf("%s\n", p);
5.     ++p;
6.     printf("%s\n", p);
7.     return 0;
8. }
```

40. Seguindo o exemplo anterior, explique qual o significado de cada um dos seguintes comandos, considerando que aparecem do lado direito de uma atribuição.

- a) (++p) : incrementa p e depois retorna o valor dessa variável
- b) (*p)
- c) (*p)+10
- d) *(p+10)
- e) &p

41. Analise o código abaixo, mostre qual será sua saída e com base na saída explique o que está realizando.

```
1. #include <stdio.h>
2. int main (int argc, char* argv[]) {
3.     int i = 0;
4.     int numeros[] = {1,2,3,4,5,6};
5.     int *p = numeros;
6.     printf ("%d", numeros[0]);
7.     for (i = 2; i < 6; i = i+2) {
8.         printf(", %d", *(p+i));
9.     }
10.    return 0;
11. }
```

42. Dada a seguinte declaração, determine quais alternativas são válidas, indicando as operações realizadas, e explique o motivo do erro das demais. Se tiver dúvidas, utilize o compilador.

```
int numeros[10], *p, x;
```

- a) `p = numeros + 1;`
- b) `p = numeros++;`
- c) `p = ++numeros;`
- d) `x = (*numeros)++;`

43. Indique qual a saída do programa abaixo:

```
1. #include <stdio.h>
2. int soma (int a, int *b);
3.
4. int main (int argc, char *argv[]){
5.     int a = 3;
6.     int *b = &a;
7.     int c = soma (a, b);
8.     printf("a = %d, b = %d, c = %d\n", a, *b, c);
9. }
10.
11. int soma (int a, int *b){
12.     a = 6;
13.     *b = 4;
14.     return a + (*b);
15. }
```

44. Implemente um programa que leia 3 nomes de até 6 caracteres, e os armazene em uma estrutura de ponteiro para ponteiro de char. Depois percorra essa estrutura imprimindo cada um dos nomes em uma linha diferente.

45. Com base no exercício anterior, represente a memória utilizada, considerando que os nomes lidos foram Carlos, Diego e Elisa. Considere também que a primeira letra do primeiro nome lido será armazenada na posição 0x0001.

46. Analise a função a seguir e escreva o que está realizando em termos dos parâmetros de entrada.

```
1. void misterio (int *a, int *b){
2.     int aux = *a;
3.     *a = *b;
4.     *b = aux;
5. }
```

47. Que tipo de variáveis são obtidas a partir das seguintes declarações?

- a) `int **a;`
- b) `char *b[3];`
- c) `_oat c;`
- d) `double *d;`

48. Considere a figura abaixo, que representa a memória utilizada por um determinado programa. O ponteiro para char `c` aponta para a primeira posição (0x0001). Escreva um trecho de código que imprima todos os endereços de memória nos quais o valor

armazenado seja a letra a. **Dica:** o operador %p na função printf() é utilizado para imprimir uma representação do endereços de memória.

0x0001	'J'
0x0002	'T'
0x0003	'A'
0x0004	'R'
0x0005	'A'
0x0006	'A'
0x0007	'B'
0x0008	'A'
0x0009	'\0'

49. Escreva uma função com o nome `imprime`, que receba um ponteiro para a struct `Pessoa` apresentada abaixo, e imprima todos os dados da pessoa, um em cada linha.

```
1. struct Pessoa {  
2.   char nome[60];  
3.   int idade;  
4.   int numero_usp;  
5. }
```

50. Desenvolva a função `main`, criando um novo registro da struct `Pessoa` do exercício anterior com os seus dados, e chame a função `imprime` que você desenvolveu.

51. Encontre o erro e corrija o algoritmo abaixo, e escreva qual a saída que ele apresenta.

```
1. #include <stdio.h>  
2. #include <stdlib.h>  
3.  
4. void surpresa (int **matriz, int l, int c) {  
5.     int i, j;  
6.     i = j = 0;  
7.     for (i = 0; i < l; i++) {  
8.         for (j = 0; j < c; j++) {  
9.             if ((i+j) % 2 == 0){  
10.                matriz[i][j] = 0;  
11.            }  
12.        }  
13.    }  
14. }  
15.  
16. int main (int argc, char *argv[]) {  
17.     int matriz [3][3] = {1,2,3,4,5,6,7,8,9};  
18.     surpresa(matriz, 3, 3);  
19.     int i, j;  
20.     for (i = 0; i < 3; i++){  
21.         for (j = 0; j < 3; j++){  
22.             printf("%d", matriz[i][j]);  
23.         }  
24.         printf("\n");  
25.     }  
26. }  
27. return 0;
```

28. }

52. Escreva uma função que recebe uma string de caracteres e uma letra e devolve um vetor de inteiros contendo as posições (índices no vetor da string) onde a letra foi encontrada) e um inteiro contendo o tamanho do vetor criado (total de letras iguais encontradas). Utilize o retorno de um vetor para retornar os índices e um ponteiro para guardar o tamanho do vetor.
53. Escrever um programa que cadastre o nome, a matrícula e duas notas de vários alunos. Em seguida imprima a matrícula, o nome e a média de cada um deles.
54. Escrever um programa que cadastre o nome, a altura, o peso, o cpf e sexo de algumas pessoas. Com os dados cadastrados, em seguida localizar uma pessoa através do seu CPF e imprimir o seu IMC.
55. Escrever um programa que cadastre vários produtos. Em seguida, imprima uma lista com o código e nome de cada produto. Por último, consulte o preço de um produto através de seu código.
56. Escreva um programa que simule contas bancárias, com as seguintes especificações:
- Ao iniciar o programa vamos criar contas bancárias para três clientes.
 - Cada conta terá o nome e o CPF do cliente associado a ela.
 - No ato da criação da conta o cliente precisará fazer um depósito inicial.
 - Após as contas serem criadas, o sistema deverá possibilitar realizações de saques ou depósitos nas contas.
 - Sempre que uma operação de saque ou depósito seja realizada, o sistema deverá imprimir o nome do titular e o saldo final da conta.

57. O método `misterio(&i,&j)` tem um problema. Qual é? Antes da chamada do método, temos a seguinte linha de comando: `int i=6, j=10;`

```
void misterio(int *p, int *q){  
    int *temp;  
    *temp = *p;  
    *p = *q;  
    *q = *temp;  
}
```

58. O que as linhas abaixo fazem?

```
int i=99,j;  
int *p;  
p = &i;  
j = *p + 100;
```

59. O que as linhas abaixo fazem?

```
int a=5, b=12;  
int *p;  
int *q;  
p = &a;  
q = &b;  
int c = *p + *q;
```

60. O que as linhas abaixo fazem?

```
int i=7, j=3;
```

```

int *p;
int **r;
p = &i;
r = &p;
c = **r + j;

```

61. Crie um vetor V com n inteiros, onde n é um valor inteiro fornecido pelo usuário. O vetor só deve ser alocado na memória depois que o usuário fornecer o valor de n.
62. Escreva um método que receba um vetor V com 10 inteiros armazenados. O método deve retornar o vetor V com o valor de cada posição multiplicado por 2. Os valores devem ser acessíveis fora do método.
63. Escreva um método que receba um vetor de inteiros e retorne o maior elemento e o menor elemento. Observe que o método deve retornar ao local da chamada os dois valores (não imprimir ao usuário). Portanto, você precisará usar passagem de parâmetro por referência, já que os métodos só podem retornar um único valor.
64. Escreva um método recursiva para determinar o menor elemento de um vetor não-ordenado de inteiros.
65. Explique o que acontece nas atribuições abaixo (dica: use desenhos):
 - (a) `p->item = q->item;`
 - (b) `p = p->prox;`
 - (c) `p = q;`
 - (d) `p->prox = q;`
 - (e) `q->ant = p;`
 - (f) `p->prox = NULL;`
 - (g) `p=p->prox->prox;`
 - (h) `p->prox = q->prox;`
66. **Objetivo:** Usar ponteiros para passar parâmetros para funções. Complete o programa 1. Este programa usa a função void troca (int *a, int *b). Esta função troca os valores apontados por a e b.

Listing 1: Programa do problema 40.

```

#include <stdio.h>
void troca ( int *a, int *b) {
    int temp ;
    temp = *a;
    *a = *b;
    /* ***** Falta um comando aqui */
} /* Fim de troca */
int main (void) {
    int x, y;
    scanf ("%d %d", &x, &y);
    troca (&x, &y);
    printf (" Troquei ----> %d %d\n", x, y);
    return 0;
}

```

67. **Objetivo:** Manipular variáveis usando ponteiros para variáveis previamente declaradas. Considere o programa 2. Procure entender o que será impresso. Procure entender a diferença na notação de ponteiros em `p2 = &j` e `*p2 = temp`.

Listing 2: Programa do problema 41.

```
#include <stdio.h>
#include <stdlib.h>
int main (void) {
    int i = 10, j = 20;
    int temp ;
    int *p1 , *p2;
    p1 = &i; /* p1 recebe endereco de i */
    p2 = &j /* p2 recebe endereco de j */
    temp = *p1; /* conteudo apontado por p1 para temp */
    *p1 = *p2; /* conteudo apontado por p2 para o apontado p1 */
    *p2 = temp ; /* conteudo apontado por p1 para p2 */
    /* O que sera impresso ??? */
    printf ("%d %d\n", i, j);
    return 0;
}
```

68. O programa 1 foi modificado levemente e virou 3. Complete os pedaços que faltam.

Listing 3: Programa do problema 42.

```
#include <stdio.h>
void Troca ( int *a, int *b) {
    int temp ;
    temp = *a; *a = *b; *b = temp ;
} /* Fim de Troca */
int main (void) {
    int x, y;
    int *px , *py;
    /* Nao se usa isto normalmente . Isto é um exercício para aprendermos ponteiros
    */
    px = &x;
    py = &y;
    /*
    * Como seria o comando scanf com px e py ao inves x e y?
    * Dica : lembre -se que era scanf ("%d %d", &x, &y);
    * Agora olhe os dois comandos acima e verifique quem e igual
    * a &x e &y.
    */
    scanf ("%d %d" , , );
    /* Como seria Troca com px e py? */
    Troca ( , );
    /* Como seria printf com px e py? */
    printf (" Troquei ----> %d %d\n", , );
    return 0;
}
```

69. O exercício mostra um exemplo de função que retorna um ponteiro. A função `achaSobrenome` retorna o ponteiro `pnome`. O que você acha que o programa imprime?

Listing 4: Programa do problema 43.

```
#include <stdio.h>

char * achaSobrenome (char nome []) {
    char * pnome ;
    int i = 0;
    while ( nome [i] != ' ' ) {
        i++;
    }
    i++;
    pnome = & nome [i];
    return pnome ;
}

int main (void) {
    char nomeCompleto [80];
    char *p;
    puts (" Entre com o seu nome e um sobrenome .");
    gets ( nomeCompleto );
    p = achaSobrenome ( nomeCompleto );
    puts (p);
    return 0;
}
```

70. Escreva um programa que declare um inteiro, um real e um char, e ponteiros para inteiro, real, e char. Associe as variáveis aos ponteiros (use &). Modifique os valores de cada variável usando os ponteiros. Imprima os valores das variáveis antes e após a modificação.
71. Escreva um programa que contenha duas variáveis inteiras. Compare seus endereços e exiba o maior endereço.
72. Escreva um programa que contenha duas variáveis inteiras. Leia essas variáveis do teclado. Em seguida, compare seus endereços e exiba o conteúdo do maior endereço.
73. Faça um programa que leia 2 valores inteiros e chame uma função que receba estas 2 variáveis e troque o seu conteúdo, ou seja, esta função é chamada passando duas variáveis A e B por exemplo e, após a execução da função, A conterá o valor de B e B terá o valor de A.
74. Faça um programa que leia dois valores inteiros e chame uma função que receba estes 2 valores de entrada e retorne o maior valor na primeira variável e o menor valor na segunda variável. Escreva o conteúdo das 2 variáveis na tela.
75. Elaborar um programa que leia dois valores inteiros (A e B). Em seguida faça uma função que retorne a soma do dobro dos dois números lidos. A função deverá armazenar o dobro de A na própria variável A e o dobro de B na própria variável B.
76. Crie um programa que contenha uma função que permita passar por parâmetro dois números inteiros A e B. A função deverá calcular a soma entre estes dois números e armazenar o resultado na variável A. Esta função não deverá possuir retorno, mas

deverá modificar o valor do primeiro parâmetro. Imprima os valores de A e B na função principal.

77. Crie um programa que contenha um array de float contendo 10 elementos. Imprima o endereço de cada posição desse array.
78. Crie um programa que contenha uma matriz de float contendo 3 linhas e 3 colunas. Imprima o endereço de cada posição dessa matriz.
79. Crie um programa que contenha um array de inteiros contendo 5 elementos. Utilizando apenas aritmética de ponteiros, leia esse array do teclado e imprima o dobro de cada valor lido.
80. Crie um programa que contenha um array contendo 5 elementos inteiros. Leia esse array do teclado e imprima o endereço das posições contendo valores pares.
81. Faça um programa que leia três valores inteiros e chame uma função que receba estes 3 valores de entrada e retorne eles ordenados, ou seja, o menor valor na primeira variável, o segundo menor valor na variável do meio, e o maior valor na última variável. A função deve retornar o valor 1 se os três valores forem iguais e 0 se existirem valores diferentes. Exibir os valores ordenados na tela.
82. Elabore uma função que receba duas strings como parâmetros e verifique se a segunda string ocorre dentro da primeira. Use aritmética de ponteiros para acessar os caracteres das strings.
83. Crie uma função que receba dois parâmetros: um array e um valor do mesmo tipo do array. A função deverá preencher os elementos de array com esse valor. Não utilize índices para percorrer o array, apenas aritmética de ponteiros.
84. Crie uma função que receba como parâmetro um array e o imprima. Não utilize índices para percorrer o array, apenas aritmética de ponteiros.
85. Considere a seguinte declaração: `int A, *B, **C, ***D;` Escreva um programa que leia a variável A e calcule e exiba o dobro, o triplo e o quádruplo desse valor utilizando apenas os ponteiros B, C e D. O ponteiro B deve ser usada para calcular o dobro, C o triplo e D o quádruplo.
86. Escreva uma função que dado um número real passado como parâmetro, retorne a parte inteira e a parte fracionária deste número. Escreva um programa que chama esta função.
Protótipo:
`void frac(float num, int* inteiro, float* frac);`
87. Implemente uma função que calcule a área da superfície e o volume de uma esfera de raio R. Essa função deve obedecer ao protótipo:
`void calc_esfera(float R, float *area, float *volume)`
A área da superfície e o volume são dados, respectivamente, por:
$$A = 4 * \pi * R^2$$
$$V = 4/3 * \pi * R^3$$
88. Escreva uma função que aceita como parâmetro um array de inteiros com N valores, e determina o maior elemento do array e o número de vezes que este elemento ocorreu no array. Por exemplo, para um array com os seguintes elementos: 5, 2, 15, 3, 7, 15, 8, 6, 15, a função deve retornar para o programa que a chamou o valor 15 e o número 3 (indicando que o número 15 ocorreu 3 vezes). A função deve ser do tipo void.
89. Implemente uma função que receba como parâmetro um array de números reais de tamanho N e retorne quantos números negativos há nesse array. Essa função deve obedecer ao protótipo:
`int negativos(float *vet, int N);`

90. Escreva um programa que declare um array de inteiros e um ponteiro para inteiros. Associe o ponteiro ao array. Agora, some mais um (+1) a cada posição do array usando o ponteiro (use *).
91. Crie uma função para somar dois arrays. Esta função deve receber dois arrays e retornar a soma em um terceiro array. Caso o tamanho do primeiro e segundo array seja diferente então a função retornará ZERO (0). Caso a função seja concluída com sucesso a mesma deve retornar o valor UM (1). Utilize aritmética de ponteiros para manipulação do array.
92. Escreva uma função que retorna o maior valor de um array de tamanho N. Escreva um programa que leia N valores inteiros, imprima o array com k elementos por linha, e o maior elemento. O valor de k também deve ser fornecido pelo usuário.
93. Escreva uma função que receba um array de inteiros V e os endereços de duas variáveis inteiras, min e max, e armazene nessas variáveis o valor mínimo e máximo do array. Escreva também uma função main que use essa função.
94. Faça um programa que possua uma função para
- ler 2 notas e retorná-las por parâmetro (chamar uma função dedicada a ler 2 notas válidas e que devolver os 2 números lidos);
 - calcular a média simples e a média ponderada e retorná-las por parâmetro, onde a segunda nota tem peso 2 media ponderada = $(n1 + n2 \cdot 2) / 3$;
95. Implemente uma função que calcule as raízes de uma equação do segundo grau do tipo $Ax^2 + Bx + C = 0$. Lembrando que:

$$X = \frac{-B \pm \sqrt{\Delta}}{2A}$$

Onde

$$\Delta = B^2 - 4AC$$

A variável A tem que ser diferente de zero.

- Se $\Delta < 0$ não existe real.
- Se $\Delta = 0$ existe uma raiz real.
- Se $\Delta \geq 0$ existem duas raízes reais.

Essa função deve obedecer ao seguinte protótipo:

int raizes(float A, float B, float C, float * X1, float * X2);

Essa função deve ter como valor de retorno o número de raízes reais e distintas da equação. Se existirem raízes reais, seus valores devem ser armazenados nas variáveis apontadas por X1 e X2.

96. Crie um programa contendo o seguinte trecho de código:

```
int a = 25;
int *pa = &a;
int b = *pa + a;
printf("%d %d %d %d %d %d\n", a, pa, &a, *pa, b, &b);
```

- a) Qual o resultado da execução do programa?
- b) Qual o significado de cada um dos valores escritos na tela?

97. Crie um programa para calcular a área e o perímetro de um hexágono. O seu programa deve implementar uma função chamada `calcula_hexagono` que calcule a área e o perímetro de um hexágono regular de lado L . A função deve obedecer o seguinte protótipo:

`void calcula_hexagono(float l, float *area, float *perimetro);`

Lembrando que a área e o perímetro de um hexágono regular são dados por:

$$A = \frac{3l^2\sqrt{3}}{2} \quad P = 6l$$

Para os cálculos, obrigatoriamente você deve utilizar as funções `sqrt` e `pow` da biblioteca `math.h`.

Em seguida crie a função principal do programa e utilize a função `calcula_hexagono` para calcular a área e o perímetro de um hexágono de lado l informado pelo usuário.

98. Escreva uma função que determine a média e a situação de um aluno em uma disciplina. A função recebe como parâmetros as três notas de um aluno (p_1 , p_2 , e p_3), seu número de faltas (`faltas`), o número total de aulas da disciplina (`aulas`) e o ponteiro para uma variável (`media`), conforme o seguinte protótipo:

`char situacao(float p1, float p2, float p3, int faltas, int aulas, float *media);`

Na variável indicada pelo ponteiro `media`, a função deve armazenar a média do aluno, calculada como a média aritmética das três provas. Além disso, a função deve retornar um caractere indicando a situação do aluno no curso, definido de acordo com o seguinte critério:

Número de Faltas	Média	Situação	Retorno
Menor ou igual a 25% do total de aulas	Maior ou igual 6,0	Aprovado	A
	Menor que 6,0	Reprovado	R
Maior que 25% do total de aulas	Qualquer	Reprovado por faltas	F

Em seguida, escreva a função principal de um programa que utiliza a função anterior para determinar a situação de um aluno. O programa deve:

- Ler do teclado três números reais e dois números inteiros, representando as notas da p_1 , p_2 e p_3 , o número de faltas e o número de aulas, respectivamente;
- Chamar a função desenvolvida na primeira questão para determinar a média e a situação do aluno na disciplina;
- Exibir a média (com apenas uma casa decimal) e a situação do aluno, isto é, "APROVADO", "REPROVADO" ou "REPROVADO POR FALTAS", dependendo do caractere retornado pela função, conforme a tabela acima.

99. Crie um programa para manipular vetores. O seu programa deve implementar uma função chamada `inverte_vetor`, que recebe como parâmetro dois vetores `V1` e `V2`, ambos de tamanho `N`. A função deve copiar os elementos de `V1` para `V2` na ordem inversa. Ou seja, se a função receber `V1 = {1,2,3,4,5}`, a função deve copiar os elementos para `V2` na seguinte ordem: `V2 = {5,4,3,2,1}`. Além disso, a função também deve retornar o maior valor encontrado em `V1`. A função deve obedecer ao seguinte protótipo:

`int inverte_vetor(int *v1, int *v2, int n);`

Em seguida, implemente no mesmo modulo outra função chamada `multiplica_escalar`, que recebe como parâmetro dois vetores `V1` e `V2` (ambos de tamanho `N`), e um número inteiro `X`. A função deve multiplicar cada um dos elementos de `V1` por `X` e armazenar os resultados em `V2`.

A função deve obedecer ao seguinte protótipo:

`void multiplica_escalar(int *v1, int *v2, int x, int n);`

Em seguida crie a função principal do programa utilizando as funções `inverte_vetor` e `multiplica_escalar` para inverter um vetor de tamanho 10 fornecido pelo usuário e em seguida multiplicar esse vetor por um escalar também fornecido pelo usuário. Por último, o programa deverá exibir o vetor resultante.

100. (Travessia de Labirinto) A grade de uns e zeros a seguir é uma representação bidimensional de um labirinto.

```
1111111111 100010000001 001010111101 111010000101 100001110100 111101010101
100101010101 110101010101 1000000000101 111111011101 100000010001
111111111111
```

Os uns representam as paredes do labirinto, e os zeros representam os quadrados (espaços) dos caminhos possíveis para cruzar o labirinto.

Há um algoritmo simples para atravessar um labirinto que garante a localização da saída (admitindo que há uma saída). Se não houver uma saída, você chegará novamente ao local de partida. Coloque sua mão direita na parede de sua direita e comece a andar para a frente. Nunca tire sua mão direita da parede. Se o labirinto dobrar para a direita, siga a parede da direita. Contanto que sua mão direita não seja retirada da parede, você acabará chegando à saída. Pode haver um caminho mais curto, mas esse garante sua saída do labirinto. Escreva uma função recursiva `travessiaLabirinto` para atravessar um labirinto. A função deve receber como argumentos um array 12-por-12 de caracteres representando o labirinto e um local de `p`: A medida que a função `travessiaLabirinto` tentar localizar uma saída do labirinto, ela deve colocar um caractere `X` em cada quadrado do caminho percorrido. A função deve exibir o labirinto depois de cada movimento para que o usuário possa ver como sua saída será encontrada.